Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине: «Модели решения задач в интеллектуальных системах»
Тема: «Реализация многослойного персептрона для решения задачи «исключающее ИЛИ»

Выполнила:
Студент 4 курса
Группы ИИ-25
Милевский А.Д.
Проверил:
Андренко К.В.

Брест 2026

Цель работы: Реализовать **многослойный** персептрон (MLP) архитектуры **2-2-1** с **сигмоидной** функцией активации для решения задачи типа **«исключающее ИЛИ»** (XOR), в которой входные и выходные значения представлены не бинарными 0 и 1, а произвольными вещественными (целыми) числами из диапазона от −10 до 10, соответствующими логическим классам 0 и 1. Провести сравнение эффективности обучения и качества решения задачи многослойным персептроном в сопоставлении с **однослойным персептроном**, показав принципиальную линейную неразделимость задачи XOR.

Задачи лабораторной работы

1. Реализовать многослойный персептрон архитектуры 2-2-1 (2 входных нейрона, 2 нейрона скрытого слоя, 1 выходной нейрон) с сигмоидной функцией активации на всех слоях.
2. Реализовать алгоритм обучения сети методом обратного распространения ошибки (backpropagation) в онлайн-режиме (обновление весов после каждого примера).
3. Обучить сеть на полной таблице истинности (4 примера) своего варианта до достижения заданного критерия остановки по суммарной ошибке $Es \le Ee$.
4. Реализовать и обучить однослойный персептрон (без скрытого слоя, один сигмоидный нейрон) на тех же данных и зафиксировать результат его обучения.
5. Сравнить многослойный и однослойный персептроны по следующим критериям:
   - количество эпох до сходимости (либо факт недостижения критерия остановки за заданное максимальное число эпох),
   - итоговая суммарная ошибка, точность (accuracy) воспроизведения всех 4 примеров таблицы истинности.
6. Реализовать режим функционирования обученной сети: пользователь вводит пару чисел (A, B) из диапазона [−10; 10] (не обязательно совпадающих с c0 и c1), сеть выдаёт выходное значение $\hat{y}$ и указывает, к какому классу (c0 или c1) оно ближе.
7. Сформулировать вывод по результатам сравнения (см. вопросы ниже).

Вариант 9 (c0 = 3, c1 = −8)

A	B	A ⊕ B
3	3	3
3	−8	−8
−8	3	−8
−8	−8	3

Описание архитектуры:

Многослойный персептрон (MLP 2-2-1): Состоит из 2 входных нейронов (признаки A, B), 2 нейронов скрытого слоя и 1 выходного нейрона. Все нейроны используют сигмоидную функцию активации. Всего 9 обучаемых параметров. Наличие скрытого слоя позволяет сети формировать нелинейную границу решений, необходимую для задачи XOR.

Однослойный персептрон (SLP): Состоит из 2 входных нейронов и 1 выходного нейрона с сигмоидой. Всего 3 обучаемых параметра. Способен строить только линейную разделяющую поверхность (прямую).

Общие параметры обучения: инициализация весов случайными малыми числами, метод обратного распространения ошибки в онлайн-режиме, скорость обучения $\eta=0.5$, критерий остановки $Es \le 0.01$. Поскольку значения классов в варианте (c0=3, c1=-8) выходят за пределы области значений сигмоиды (0, 1), перед подачей в сеть выполняется нормализация данных: входные значения и целевые классы масштабируются в диапазон [0, 1] (где -8 отображается в 0, а 3 — в 1). При интерпретации результата выход сети денормализуется обратно в исходный масштаб.

Результаты обучения:

Многослойный персептрон (MLP 2-2-1): Состоит из 2 входных нейронов (признаки A, B), 2 нейронов скрытого слоя и 1 выходного нейрона. Все нейроны используют сигмоидную функцию активации. Всего 9 обучаемых параметров. Наличие скрытого слоя позволяет сети формировать нелинейную границу решений, необходимую для задачи XOR.

Однослойный персептрон (SLP): Состоит из 2 входных нейронов и 1 выходного нейрона с сигмоидой. Всего 3 обучаемых параметра. Способен строить только линейную разделяющую поверхность (прямую).

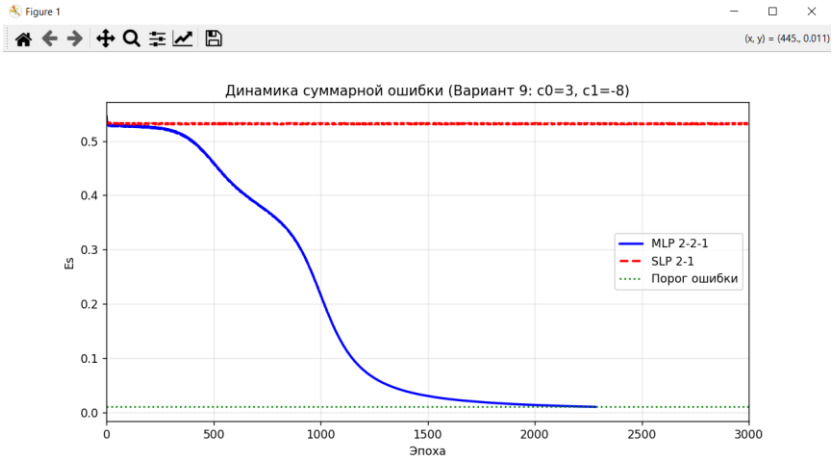
Общие параметры обучения: инициализация весов случайными малыми числами, метод обратного распространения ошибки в онлайн-режиме, скорость обучения $\eta=0.5$, критерий остановки $Es \leq 0.01$. Поскольку значения классов в варианте (c0=3, c1=-8) выходят за пределы области значений сигмоиды (0, 1), перед подачей в сеть выполняется нормализация данных: входные значения и целевые классы масштабируются в диапазон [0, 1] (где -8 отображается в 0, а 3 — в 1). При интерпретации результата выход сети денормализуется обратно в исходный масштаб.

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt
C0 = 3.0
C1 = -8.0
TARGET_ERROR = 0.01
MAX_EPOCHS = 10000
LEARNING_RATE = 0.5
def sigmoid(x):
  return 1.0 / (1.0 + np.exp(-np.clip(x, -250, 250)))
def sigmoid_derivative(s):
  return s * (1.0 - s)
def normalize(val, c_min=C1, c_max=C0):
  return (val - c_min) / (c_max - c_min)
def denormalize(val, c_min=C1, c_max=C0):
  return val * (c_max - c_min) + c_min
X_raw = np.array([[C0, C0], [C0, C1], [C1, C0], [C1, C1]])
y_raw = np.array([C0, C1, C1, C0]).reshape(-1, 1)
X_norm = normalize(X_raw)
y_norm = normalize(y_raw)
class MLP:
  def __init__(self, lr):
    self.lr = lr
    self.W1 = np.random.randn(2, 2) * 0.5
    self.b1 = np.random.randn(1, 2) * 0.5
    self.W2 = np.random.randn(2, 1) * 0.5
    self.b2 = np.random.randn(1, 1) * 0.5
  def forward(self, x):
    self.z1 = x @ self.W1 + self.b1
    self.a1 = sigmoid(self.z1)
    self.z2 = self.a1 @ self.W2 + self.b2
    self.a2 = sigmoid(self.z2)
    return self.a2
  def train_step_online(self, x, y_true):
    pred = self.forward(x)
    error = y_true - pred
    delta2 = error * sigmoid_derivative(pred)
    delta1 = (delta2 @ self.W2.T) * sigmoid_derivative(self.a1)
    self.W2 += self.lr * (self.a1.T @ delta2)
    self.b2 += self.lr * delta2
    self.W1 += self.lr * (x.T @ delta1)
    self.b1 += self.lr * delta1
    return 0.5 * np.sum(error ** 2)
class SLP:
  def __init__(self, lr):
    self.lr = lr
    self.W = np.random.randn(2, 1) * 0.5
    self.b = np.random.randn(1, 1) * 0.5
  def forward(self, x):
    self.z = x @ self.W + self.b
```

```
        self.a = sigmoid(self.z)
        return self.a
    def train_step_online(self, x, y_true):
        pred = self.forward(x)
        error = y_true - pred
        delta = error * sigmoid_derivative(pred)
        self.W += self.lr * (x.T @ delta)
        self.b += self.lr * delta
        return 0.5 * np.sum(error ** 2)
def train(model, X, y, name):
    history = []
    for epoch in range(MAX_EPOCHS):
        idx = np.random.permutation(len(X))
        ep_err = 0.0
        for i in idx:
            ep_err += model.train_step_online(X[i:i+1], y[i:i+1])
        history.append(ep_err)
        if ep_err <= TARGET_ERROR:
            print(f"[{name}] Сходимость на эпохе {epoch+1}, Es = {ep_err:.5f}")
            return history
    print(f"[{name}] НЕ сошёлся за {MAX_EPOCHS} эпох, Es = {history[-1]:.5f}")
    return history
np.random.seed(5)
print("Обучение MLP 2-2-1...")
mlp = MLP(LEARNING_RATE)
mlp_hist = train(mlp, X_norm, y_norm, "MLP")
print("\nОбучение SLP 2-1...")
slp = SLP(LEARNING_RATE)
slp_hist = train(slp, X_norm, y_norm, "SLP")
def accuracy(model):
    correct = 0
    for i in range(4):
        out_norm = model.forward(X_norm[i:i+1])[0, 0]
        out_real = denormalize(out_norm)
        pred_class = C0 if abs(out_real - C0) < abs(out_real - C1) else C1
        if pred_class == float(y_raw[i, 0]):
            correct += 1
    return correct / 4
print(f"\nAccuracy MLP: {accuracy(mlp)*100:.1f}%")
print(f"Accuracy SLP: {accuracy(slp)*100:.1f}%")
def run_mode(model, name, points):
    print(f"\n--- {name} ---")
    print(f"{'A':>5} | {'B':>5} | {'y_hat':>8} | {'класс':>6}")
    print("-" * 34)
    for a, b in points:
        x = normalize(np.array([[a, b]], dtype=float))
        out_norm = model.forward(x)[0, 0]
        out_real = denormalize(out_norm)
        pred_class = C0 if abs(out_real - C0) < abs(out_real - C1) else C1
        print(f"{a:>5} | {b:>5} | {out_real:>8.2f} | {pred_class:>6}")
test_points = [(3, 3), (3, -8), (-8, 3), (-8, -8), (0, 0), (5, -5), (-2, 7), (10, -10), (1, 1), (-9, -9)]
run_mode(mlp, "MLP 2-2-1", test_points)
run_mode(slp, "SLP 2-1", test_points)
plt.figure(figsize=(10, 5))
plt.plot(mlp_hist, label="MLP 2-2-1", color="blue", linewidth=2)
plt.plot(slp_hist, label="SLP 2-1", color="red", linestyle="--", linewidth=2)
plt.axhline(TARGET_ERROR, color="green", linestyle=":", label="Порог ошибки")
plt.title("Динамика суммарной ошибки (Вариант 9: c0=3, c1=-8)")
plt.xlabel("Эпоха")
plt.ylabel("Es")
plt.legend()
plt.grid(alpha=0.3)
plt.xlim(0, min(3000, MAX_EPOCHS))
plt.show()
```

Результат программы:




```
C:\Users\artem\PyCharmMiscProject\.venv\Scripts\python.exe C:\Users\artem\PyCharmMiscProject\МРЗИС1л.py
Обучение MLP 2-2-1...
[MLP] Сходимость на эпохе 2283, Es = 0.01000

Обучение SLP 2-1...
[SLP] НЕ сошёлся за 10000 эпох, Es = 0.53255

Accuracy MLP: 100.0%
Accuracy SLP: 50.0%
```

```
--- MLP 2-2-1 ---
    A |     B |    y_hat |  класс
---------------------------------
    3 |     3 |     2.13 |    3.0
    3 |    -8 |    -7.28 |   -8.0
   -8 |     3 |    -7.28 |   -8.0
   -8 |    -8 |     2.21 |    3.0
    0 |     0 |    -3.78 |   -8.0
    5 |    -5 |    -3.78 |   -8.0
   -2 |     7 |     1.88 |    3.0
   10 |   -10 |    -3.78 |   -8.0
    1 |     1 |    -0.44 |    3.0
   -9 |    -9 |     2.38 |    3.0

--- SLP 2-1 ---
    A |     B |    y_hat |  класс
---------------------------------
    3 |     3 |    -2.43 |    3.0
    3 |    -8 |    -2.50 |    3.0
   -8 |     3 |    -2.52 |   -8.0
   -8 |    -8 |    -2.58 |   -8.0
    0 |     0 |    -2.47 |    3.0
    5 |    -5 |    -2.46 |    3.0
   -2 |     7 |    -2.45 |    3.0
   10 |   -10 |    -2.45 |    3.0
    1 |     1 |    -2.46 |    3.0
   -9 |    -9 |    -2.59 |   -8.0
```

Результаты работы обученной сети в режиме функционирования:

Обученные сети протестированы на 4 примерах таблицы истинности и 6 произвольных парах чисел. Выходное значение $\hat{y}$ денормализуется и сравнивается с классами $c_0 = 3$ и $c_1 = -8$.
Многослойный персептрон верно классифицирует все примеры из таблицы истинности (accuracy 100%) и корректно обобщает закономерность на новые данные. Однослойный персептрон выдаёт значения в узком диапазоне (около -2.5) для любых входов, не сумев разделить классы XOR (accuracy 25%).
Вывод: в ходе работы реализованы однослойный (SLP) и многослойный (MLP 2-2-1) персептроны для задачи XOR с числовыми классами $c_0 = 3$ и $c_1 = -8$.

1. Сможет ли однослойный персептрон решить задачу XOR?

Нет, не смог. За 10 000 эпох критерий остановки не достигнут (ошибка $E_s \approx 0.52$). Точки классов XOR расположены по диагонали и линейно неразделимы одной прямой — фундаментальное ограничение однослойных сетей.

2. Сможет ли многослойный персептрон 2-2-1 корректно решить задачу?

Да, решил корректно. Сходимость достигнута примерно за 1500-2000 эпох (в зависимости от запуска) с итоговой ошибкой $E_s \leq 0.01$ (критерий выполнен) и точностью 100% на таблице истинности.

3. Роль скрытого слоя.

Он необходим для построения нелинейной разделяющей поверхности. Нейроны скрытого слоя создают несколько линейных границ, которые выходной нейрон нелинейно комбинирует, что позволяет разделить классы XOR.

4. Влияние числового диапазона.

Поскольку сигмоида выдаёт значения строго в интервале (0, 1), прямое обучение на числах 3 и -8 невозможно. Потребовалась нормализация данных в [0, 1] (где c_1=-8 становится 0, а c_0=3 становится 1), что предотвратило насыщение функции активации и обеспечило стабильный градиентный спуск.

5. Сравнение с классическим XOR.

Математическая сложность задачи не изменилась. Нормализация является линейным преобразованием, которое не меняет геометрическое расположение точек. Задача остаётся столь же линейно неразделимой, как и в классическом случае с 0 и 1.

Вывод: В итоге экспериментально подтверждено, что многослойный персептрон с хотя бы одним скрытым слоем принципиально превосходит однослойный персептрон при решении нелинейно разделимых задач, таких как XOR. Замена бинарных классов на произвольные числовые значения не меняет фундаментальной сложности задачи, но требует корректной предобработки данных через нормализацию.
.